



Affordmed Interview Question Bank

Topic-Wise Questions with Sample Answers — Mock Interview & Student Prep Reference

How to use this document: sample answers are templates, not scripts. Students should adapt every bracketed [placeholder] to their own real project, internship, and experience. For the Round 1 Assessment section specifically, students must answer using their OWN submitted code — interviewers will know the difference.

Contents

1. Opening — Tell Me About Yourself & Warm-Up
2. Round 1 Assessment — Real-Time Implementation & System Design
3. Projects
4. Internships
5. Database & SQL
6. Backend Engineering
7. Frontend Engineering
8. Data Structures & Algorithms
9. OOP & Software Engineering
10. Behavioral & HR Closing Questions
11. Glassdoor Interview Questions — Real Candidate Reports
12. Final Round Logistics & Live Coding Prep



1. Opening — Tell Me About Yourself & Warm-Up

These set the tone for the whole interview. Keep answers under 90 seconds, and always land on why you're a fit for this specific role.

Q1. Tell me about yourself.

Sample Answer: I'm a final-year [Branch] student at [College]. I've spent the last two years going deep into full-stack development — backend with Node/Express and databases, frontend with React. My biggest project is [Project Name], where I [one-line impact, e.g. 'built a real-time notification system for 500+ users']. I also [internship/certification, if any]. I'm looking for a role like this one at Affordmed because it lets me work across the stack on real production systems, which is exactly the kind of work I enjoy most.

Q2. Walk me through your resume.

Sample Answer: I'll start from my strongest, most recent project rather than reading top to bottom: [Project], where I used [stack] to solve [problem]. Before that, I built [second project] to learn [skill]. Academically, I've focused on [DSA/DB/web dev] coursework, and I've been consistently practicing [platform, e.g. LeetCode] to stay sharp on fundamentals. That combination is why I feel ready for a developer role today.

Q3. Why do you want to work at Affordmed?

Sample Answer: Affordmed builds real, production-grade systems — which I saw firsthand in the Round 1 assessment, where the problems felt like genuine engineering work, not textbook exercises. I want to work somewhere my code has to survive contact with real scale and real constraints, and this role's mix of backend and frontend work matches exactly the kind of full-stack growth I'm looking for early in my career.

Q4. What are your strengths and weaknesses?

Sample Answer: My strength is that I don't stop at 'it works' — in my [project], I went back and added indexing after noticing a query was slow, even though it wasn't required. My weakness is that I can spend too long polishing a small piece instead of shipping and iterating; I've been actively working on this by timeboxing tasks and setting myself deadlines for 'good enough, ship it' moments.

Q5. Where do you see yourself in 3–5 years?

Sample Answer: In the next couple of years I want to go deep on backend systems — databases, APIs, and scale — while staying comfortable on the frontend. Longer term, I'd like to be the engineer people come to when something is slow or breaking in production, because I understand the system end to end, not just the part I wrote.

Q6. Why should we hire you over other candidates?

Sample Answer: Beyond just clearing the assessment, I can explain every decision I made in it — why I chose that index, why I used a queue for the bulk notify problem, why greedy fails on the scheduling problem. That's the difference between having written code and actually understanding it, and I think that understanding is what makes someone productive quickly on a real team.



2. Round 1 Assessment — Real-Time Implementation & System Design

This section is a direct defense of the take-home assessment. Students should answer these using their OWN submitted code and decisions, not the generic version below — the sample answers are templates to adapt, not scripts to memorise.

Q1. Walk me through your Logging Middleware. Why these specific fields?

Sample Answer: I built Log(stack, level, package, message) as a small reusable function that posts to the evaluation service every time it's called. I chose 'stack' to separate frontend/backend events, 'level' to convey severity (debug through fatal), and 'package' to pinpoint which module raised it — together they let someone debugging in production filter noise instantly instead of reading every line.

Q2. What happens if your log server is unreachable — does your app hang, retry, or drop the log?

Sample Answer: I made the log call fire-and-forget — it's sent asynchronously and I don't await or block on its response, so a slow or unreachable log server can never freeze or crash the actual feature. In a production version, I'd add a short timeout and a local fallback queue so logs aren't silently lost, but the core app must keep functioning either way.

Q3. Walk me through one REST endpoint you designed end-to-end.

Sample Answer: For GET /notifications, the request takes optional query params like limit, page, and notification_type. The response returns a JSON array of notification objects with id, type, message, and timestamp, plus a 200 status on success. I chose this shape because it supports both pagination and filtering without needing separate endpoints, which keeps the API surface small and predictable.

Q4. Why did you choose polling / WebSockets / SSE for real-time notifications?

Sample Answer: I chose [X] because [reasoning — e.g. 'SSE gives one-way server-to-client push without the complexity of a full WebSocket connection, which fits a notification feed that never needs the client to send data back in real time']. Polling would waste requests when nothing's changed, and WebSockets would be over-engineering for a one-directional feed.

Q5. Explain the slow query problem you fixed. Why was it slow, and what did you change?

Sample Answer: The original query filtered on studentID and isRead with no supporting index, then sorted by createdAt — at 5 million rows, that forces a full table scan followed by an expensive sort. I added a composite index on (studentID, isRead, createdAt), with studentID first since it's the equality filter, so the database can jump straight to the relevant rows already in sorted order.

Q6. Why is 'index every column to be safe' bad advice?

Sample Answer: Every additional index has to be updated on every insert, update, and delete, so more indexes mean slower writes and more storage — and past a point, the query planner has more options to weigh, which can even slow down reads. Indexes should be added deliberately, based on the actual queries the application runs, not applied blanket across every column.

Q7. How did you make your bulk-notify redesign reliable when 200 of 50,000 emails failed?

Sample Answer: I decoupled the DB write from the email send so one failing doesn't roll back or block the other, and made the whole job idempotent by tracking which student IDs were already notified — so re-running it after a partial failure only retries the 200 that failed, instead of re-sending to everyone. In a full production version I'd push this through a queue with retry-with-backoff.

Q8. Why did you use a heap for the Priority Inbox instead of sorting the list each time?

Sample Answer: Sorting the entire list on every new notification is $O(n \log n)$ every time, which gets expensive as the feed grows. A heap keyed on priority (weight + recency) lets me maintain the top-N in $O(\log n)$ per insert, and I only re-sort the visible top-N for display, not the whole dataset — that's the right trade-off when new items keep streaming in.

**Q9. (Backend-only) Why is the Vehicle Maintenance Scheduler a Knapsack problem, and why does greedy fail?**

Sample Answer: Each task has a duration (weight) and an impact score (value), and there's a fixed mechanic-hour budget (capacity) — that's exactly 0/1 Knapsack. Greedy-by-impact fails because picking the highest-impact task first can use up hours that would've unlocked a much better combination of two or three smaller tasks; I used dynamic programming, building up the best achievable impact for every hour budget from smaller subproblems.

Q10. How would you test that your Logging Middleware actually works before relying on it elsewhere?

Sample Answer: I'd call it directly with a few obvious cases first — a valid debug log, an invalid level value, a missing field — and confirm each produces the response the test API documents. Only once I trusted it in isolation would I start wiring it into the rest of my code, rather than debugging both at once.

Q11. What would happen if your app tried to send hundreds of log calls in a very short burst?

Sample Answer: Since each call is fire-and-forget, they'd all fire off independently, which could flood the log server or hit a rate limit. In a production version I'd batch logs and flush them on an interval or size threshold instead of sending one network call per log line, to keep the overhead predictable under load.

Q12. Why did you structure your REST endpoints and JSON schemas the way you did in Stage 1?

Sample Answer: I kept the resource naming consistent and predictable — plural nouns, nested routes for relationships — and kept each JSON response to the fields the frontend would actually need, instead of dumping the entire database row. That makes the contract easier for a frontend developer to consume without needing to ask me questions.

Q13. Why did you choose the database you chose in Stage 2, specifically for this notification system?

Sample Answer: I chose [SQL/NoSQL] because [reasoning — e.g. 'notifications have a clear, consistent shape and a real relationship to a student record, which fits a relational schema with a foreign key' or 'the write volume and flexible message structure fit a document store better than rigid tables']. I'd also mention what I'd reconsider if the access patterns changed significantly.

Q14. Write the query to find all students who received a placement notification in the last 7 days.

Sample Answer: `SELECT DISTINCT studentID FROM notifications WHERE notificationType = 'Placement' AND createdAt >= NOW() - INTERVAL 7 DAY.` I'd explain that filtering on notificationType and a recent date range is exactly the kind of query my composite index needs to support efficiently at scale, and that DISTINCT avoids counting the same student twice if they got multiple placement notifications.

Q15. What does EXPLAIN or EXPLAIN ANALYZE actually tell you about a query, and did you use it?

Sample Answer: It shows the database's execution plan — whether it's doing a full table scan or using an index, the estimated number of rows examined, and the join/sort strategy chosen. I'd use it before and after adding my index to prove the fix actually changed the plan, rather than just assuming the index helped.

Q16. Besides indexing, what else did you consider for Stage 4's caching problem, and how would you invalidate the cache?

Sample Answer: I considered pagination (never fetching more than the current page needs) alongside caching, since the two solve slightly different problems — caching reduces repeated identical reads, pagination reduces the size of any single read. For invalidation, I'd clear or update the specific cached key the moment a new notification is written for that student, rather than clearing the whole cache indiscriminately.

Q17. In Stage 5, why not just retry the entire bulk-notify job from scratch if part of it fails?

Sample Answer: Re-running the whole job would re-send emails and push notifications to the 49,800 students who already succeeded, which is wasteful at best and confusing or annoying to users at worst. Tracking which student IDs completed lets a retry target only the 200 that actually failed.

Q18. How exactly did you combine 'weight' (placement > result > event) and recency to rank the Priority Inbox in Stage 6?



Sample Answer: I assigned each notification type a numeric weight (e.g. placement=3, result=2, event=1), then combined it with recency — for example, weight as the primary sort key and timestamp as the tiebreaker, or a single combined score like weight times a decay factor based on age. I'd be ready to justify why I picked that specific formula over alternatives.

Q19. (Full Stack only) How did you distinguish read vs. unread notifications in your Stage 7 frontend, and why Material UI?

Sample Answer: I tracked read/unread as a boolean per notification, driving both a visual style difference (e.g. bold text or a dot indicator for unread) and the filtering logic for the Priority Inbox. Material UI was specified as the required styling approach in the brief, so I used its components directly rather than reaching for a different library, to stay within the stated constraints.

Q20. Across all seven stages, what's the single biggest risk or weakness in what you actually submitted?

Sample Answer: I'd point to one honest gap myself — for example, 'my caching invalidation is time-based rather than event-based, so there's a small window where a student could see stale data' — rather than waiting for the interviewer to find it. Naming your own weak spot first shows self-awareness, which lands better than being caught off guard.

Q21. If you had one more hour on this assessment, what would you add or change?

Sample Answer: I'd name something specific and technically grounded — for example, adding proper request validation on the API inputs, writing a couple of unit tests for the Priority Inbox logic, or replacing my time-based cache invalidation with an event-driven one. A vague answer like 'polish the UI' is much weaker than naming a concrete engineering gap.



3. Projects

Use the 5-block framework from the workshop: Name & Purpose → Your Role → Tech Stack → Impact/Result → Challenge Faced. Never let an interviewer catch you describing a line of code you can't explain.

Q1. Tell me about your most challenging project.

Sample Answer: [Project Name] was the hardest because [specific technical reason, e.g. 'it needed real-time updates across multiple clients']. I built it using [stack], and the hardest part was [specific challenge, e.g. 'keeping the UI in sync without excessive re-fetching'], which I solved by [specific solution]. It taught me that the hard part of software is rarely the first version — it's making it reliable.

Q2. What was your specific contribution in this team project?

Sample Answer: I owned the [backend/database/specific feature] end to end — I designed the schema, built the APIs for [feature], and integrated them with the frontend my teammate built. I also [additional contribution, e.g. 'set up the deployment' or 'wrote the test cases']. I can walk through any part of that code in detail.

Q3. What would you do differently if you rebuilt this project today?

Sample Answer: I'd add proper indexing from day one instead of adding it after noticing slowness, and I'd introduce a caching layer for the read-heavy endpoints. I'd also write tests earlier in the process rather than after the feature was 'done' — I learned the hard way that untested code breaks quietly.

Q4. How did you handle a bug you couldn't solve for a long time?

Sample Answer: I hit a bug where [specific example, e.g. 'state updates weren't reflecting in the UI']. After trying obvious fixes, I stepped back and added logging at every stage of the data flow to see exactly where it diverged from what I expected, rather than guessing. It turned out to be [root cause]. Since then, I reach for structured logging much earlier instead of guessing-and-checking.

Q5. What's the architecture of your project?

Sample Answer: It follows a [MVC / layered] structure: a [React] frontend talks to a REST API built with [Express/Django/etc], which sits in front of a [PostgreSQL/MongoDB] database. I kept business logic in the controller/service layer, not in the routes, so it's testable independent of the HTTP layer.

Q6. How did you test your project?

Sample Answer: I used [manual testing via Postman/Insomnia, and/or unit tests with Jest] for the core business logic, especially anything involving calculations or data transformations. For the frontend, I tested manually across both desktop and mobile views since UI bugs are often layout-specific and hard to catch with unit tests alone.

Q7. How did you handle version control and collaboration in this project?

Sample Answer: We used Git with feature branches and pull requests, even though it was just the two of us — it forced us to review each other's code before merging, which caught bugs early. I committed at logical milestones with descriptive messages, not one giant commit at the end, which made it much easier to track down when a specific bug was introduced.



4. Internships

If a student has no internship, they should pivot honestly to a substantial personal or academic project and frame the same lessons — interviewers care about the learning, not the job title.

Q1. Tell me about your internship experience.

Sample Answer: I interned at [Company] as a [role] for [duration], where I worked on [specific feature/task] as part of a team of [size]. My main contribution was [specific outcome, e.g. 'building an internal dashboard that reduced manual reporting time']. It was my first experience working inside an existing, large codebase rather than starting from scratch.

Q2. What's the biggest thing you learned from your internship?

Sample Answer: The biggest shift was realising how much of real engineering is about not breaking things that already work. In college projects, I could rewrite anything; at the internship, I had to understand existing code, follow its conventions, and think through the blast radius of every change before making it.

Q3. Describe a piece of feedback you received and how you acted on it.

Sample Answer: My mentor pointed out that my pull requests were too large to review easily. After that, I started breaking work into smaller, single-purpose commits and PRs, which made reviews faster and made it easier for me to catch my own mistakes before submitting.

Q4. Describe a time you had to learn a new technology quickly on the job.

Sample Answer: I was asked to add a feature using [technology I hadn't used before], with about a week to deliver. I spent the first day going through official docs and a small sandbox project before touching the real codebase, which meant I wrote less code overall but far less of it needed to be redone.

Q5. How is working in a real team different from academic projects?

Sample Answer: In college, most projects are short and disappear after submission. At work, code has to be maintained by other people long after I write it — so naming, comments, and consistency matter much more than they did for a project graded once and forgotten.

Q6. What did you find most challenging in your internship?

Sample Answer: Estimating how long a task would actually take was harder than expected — I consistently underestimated the time needed for edge cases and testing, not just the 'happy path' implementation. I've since started padding my estimates and explicitly listing edge cases before starting a task.



5. Database & SQL

Interviewers use SQL questions to gauge real understanding, not memorised syntax — always explain the 'why', not just the query.

Q1. What's the difference between SQL and NoSQL, and when would you choose each?

Sample Answer: SQL databases enforce a fixed schema and relationships, which suits data with clear structure and where consistency matters, like financial or user account data. NoSQL databases (like MongoDB) are schema-flexible and scale horizontally more easily, which suits data that varies in shape or needs very high write throughput, like logs or activity feeds.

Q2. Explain normalization and why it matters.

Sample Answer: Normalization is organizing tables to reduce data duplication — for example, storing a student's department in one students table instead of repeating it on every notification row. It matters because duplicated data can go out of sync (update one copy, forget the other) and wastes storage; the trade-off is that heavily normalized schemas need more joins to read data back.

Q3. What are the ACID properties?

Sample Answer: Atomicity means a transaction either fully completes or fully rolls back — no partial writes. Consistency means a transaction only moves the database from one valid state to another. Isolation means concurrent transactions don't interfere with each other's intermediate states. Durability means once committed, data survives even a crash right after.

Q4. Explain entity, referential, and domain integrity.

Sample Answer: Entity integrity means every row is uniquely identifiable — enforced by a NOT NULL, unique primary key. Referential integrity means a foreign key must point to a row that actually exists in the parent table — enforced by FOREIGN KEY constraints. Domain integrity means every value respects its column's type, format, and range — enforced by data types and CHECK constraints.

Q5. What's the difference between a correlated and non-correlated subquery? Give an example.

Sample Answer: A non-correlated subquery runs once, independent of the outer query — like finding employees earning more than the overall average salary. A correlated subquery references a column from the outer row and re-runs once per row — like finding employees earning more than their own department's average, where the inner query needs e1.dept_id from the outer row each time.

Q6. How do you find the Nth highest salary?

Sample Answer: One way: `SELECT DISTINCT salary FROM employee ORDER BY salary DESC LIMIT 1 OFFSET N-1`. Another, more portable way uses a correlated subquery: find salaries where exactly N-1 distinct salaries are greater. I'd pick the LIMIT/OFFSET version for simplicity, but I know the correlated version works on databases without LIMIT support.

Q7. Explain indexing. How does a composite index work, and why does column order matter?

Sample Answer: An index is a separate, sorted structure the database uses to avoid scanning every row. A composite index covers multiple columns together — column order matters because the index is only useful for filtering on a prefix of its columns in order; an index on (studentID, isRead) helps a query filtering on studentID alone or on studentID and isRead together, but not on isRead alone.

Q8. Explain an ER diagram for a simple Students — Notifications system.

Sample Answer: STUDENTS has studentID as its primary key, plus name, email, and department. NOTIFICATIONS has notificationID as its primary key, plus a studentID foreign key pointing back to STUDENTS, a type (enum), a message, and a createdAt timestamp. The relationship is one-to-many: one student can have many notifications, but each notification belongs to exactly one student.



6. Backend Engineering

Expect follow-ups that push past definitions into trade-offs — 'why this and not that' is the real question behind most of these.

Q1. What is REST, and what makes an API RESTful?

Sample Answer: REST is an architectural style where resources are addressed by URLs and manipulated using standard HTTP methods. An API is RESTful when it's stateless (no session stored between requests), uses predictable resource-based URLs like `/notifications/:id`, and uses HTTP methods and status codes meaningfully rather than stuffing everything into POST with a 200.

Q2. Explain middleware in Express/Node.

Sample Answer: Middleware is a function that runs between the incoming request and the final route handler, with access to modify the request/response or stop the chain entirely. I use it for cross-cutting concerns like logging, authentication checks, and error handling, so I don't repeat that logic in every route.

Q3. How do you handle errors in a Node.js/Express app?

Sample Answer: I use a centralized error-handling middleware at the end of the middleware chain, so any route can just call `next(error)` and have it handled consistently — logged properly and returned with the right status code and message — instead of each route writing its own `try/catch` and response formatting.

Q4. What's the difference between authentication and authorization?

Sample Answer: Authentication answers 'who are you' — verifying identity, typically via a token or session. Authorization answers 'what are you allowed to do' — checking permissions once identity is known. A request can be authenticated but still unauthorized, like a logged-in student trying to access an admin-only endpoint.

Q5. What is caching, and how would you implement it — Redis vs in-memory?

Sample Answer: Caching stores a copy of expensive-to-compute or frequently-read data somewhere faster to access than the source of truth. In-memory caching lives inside a single server process — fast but not shared across multiple server instances and lost on restart. Redis is a separate shared cache — slightly slower than in-memory, but consistent across all servers and survives individual server restarts.

Q6. How would you design an API to handle 10x traffic overnight?

Sample Answer: First I'd identify the actual bottleneck — usually the database, not the API layer itself. I'd add caching for read-heavy endpoints, add pagination if it isn't there already, consider a read replica for the database, and add rate limiting to protect against abuse. I'd avoid just 'adding more servers' without knowing where the real bottleneck is.

Q7. What's the difference between PUT, PATCH, and POST?

Sample Answer: POST creates a new resource. PUT replaces an entire existing resource with the given data. PATCH applies a partial update, changing only the fields provided. Using PATCH to mark one notification as read, for example, is more correct than PUT, since I'm not replacing the whole notification object.

Q8. Explain rate limiting and why an API might need it.

Sample Answer: Rate limiting caps how many requests a client can make in a given time window, to protect the backend from being overwhelmed — whether from a bug in a client, a traffic spike, or a malicious actor. Without it, one misbehaving client could degrade the service for everyone else.



7. Frontend Engineering

JavaScript fundamentals questions are where many otherwise-strong candidates lose points — practice explaining these out loud, not just reading them.

Q1. Explain closures with an example.

Sample Answer: A closure is a function that remembers the variables from the scope it was created in, even after that outer function has returned. For example, a counter function that returns an inner increment function — the inner function keeps access to the counter variable even though the outer function already finished executing.

Q2. What is the event loop?

Sample Answer: JavaScript is single-threaded, so the event loop is what lets it handle async work without blocking. Synchronous code runs first on the call stack; async callbacks (from timers, promises, or I/O) wait in queues and only run once the call stack is empty. Microtasks like promise callbacks run before macrotasks like setTimeout.

Q3. What are React hooks? Explain useState and useEffect.

Sample Answer: Hooks let function components use state and lifecycle features that used to require class components. useState gives a component a piece of state and a setter function that triggers a re-render when called. useEffect runs side effects — like fetching data — after render, and can re-run when specific dependencies change, controlled by its dependency array.

Q4. What's the difference between state and props?

Sample Answer: State is data owned and managed inside a component, which can change over time and triggers a re-render when updated. Props are data passed into a component from its parent — read-only from the receiving component's perspective. State is 'mine', props are 'given to me'.

Q5. What is the virtual DOM and why does React use it?

Sample Answer: The virtual DOM is an in-memory representation of the UI. When state changes, React builds a new virtual DOM tree, compares it to the previous one, and updates only the real DOM nodes that actually changed, instead of re-rendering the whole page — this makes UI updates much faster than direct DOM manipulation.

Q6. TypeScript: interface vs type — when would you use each?

Sample Answer: Both describe the shape of an object, and for simple cases they're interchangeable. I reach for interface when defining the shape of an object or a class contract, especially if it might need to be extended later, since interfaces support declaration merging. I reach for type when I need unions, intersections, or more complex type expressions that interfaces can't express.

Q7. What's the difference between Next.js and React?

Sample Answer: React is a library for building UI components. Next.js is a framework built on top of React that adds file-based routing, server-side rendering, and static site generation out of the box. I'd reach for Next.js when I need SEO-friendly server rendering or built-in routing, and plain React when I just need a client-side app or an embedded widget.

Q8. How do you optimize a React app's performance?

Sample Answer: I'd start by checking for unnecessary re-renders using React DevTools, then apply memoization (useMemo, useCallback, React.memo) only where profiling shows it actually helps. I'd also lazy-load routes and heavy components with dynamic imports, and make sure list rendering uses stable keys instead of array indexes.



8. Data Structures & Algorithms

Interviewers care more about your reasoning process than a memorized solution — always state the brute-force approach first, then optimize out loud.

Q1. How do you decide which data structure to use for a given problem?

Sample Answer: I look at the operations the problem needs most often — if I need fast lookups, a hash map; if I need order maintained with fast insertion at both ends, a deque; if I repeatedly need the maximum or minimum from a changing set, a heap. The dominant operation in the problem usually points directly at the right structure.

Q2. Explain time and space complexity with Big-O — give an example.

Sample Answer: Big-O describes how an algorithm's runtime or memory use grows as input size grows, ignoring constant factors. A linear search through an array is $O(n)$ — double the array, double the worst-case comparisons. Binary search on a sorted array is $O(\log n)$ — doubling the array only adds one more comparison step.

Q3. Explain the sliding window pattern with an example.

Sample Answer: Sliding window avoids recomputing a result from scratch for every subarray by incrementally adjusting a window's boundaries. For example, finding the maximum sum of any k consecutive elements: instead of summing each window from scratch, I subtract the element leaving the window and add the element entering it, turning an $O(n \cdot k)$ approach into $O(n)$.

Q4. What is a 0/1 Knapsack problem, and how do you solve it?

Sample Answer: Given items each with a weight and a value, and a maximum capacity, pick the subset of items maximizing total value without exceeding capacity, where each item can only be used once. I solve it with dynamic programming: building a table where each cell represents the best achievable value using the first i items and a capacity of w , built up from smaller subproblems.

Q5. Explain how a heap works and where you'd use one.

Sample Answer: A heap is a tree-based structure where the parent is always smaller (min-heap) or larger (max-heap) than its children, letting you access the min/max in $O(1)$ and insert or remove it in $O(\log n)$. I'd use one anywhere I need a running 'top- k ' or a priority-based queue, like a task scheduler or the Priority Inbox problem from the assessment.

Q6. What is recursion, and what's a common pitfall?

Sample Answer: Recursion is a function calling itself on a smaller version of the same problem until it hits a base case. The most common pitfall is a missing or incorrect base case, which causes infinite recursion and a stack overflow; the second most common is redoing the same subproblem repeatedly without memoization, which blows up runtime exponentially.



9. OOP & Software Engineering

These questions test whether concepts are just memorized definitions or genuinely understood — always follow a definition with a concrete code example.

Q1. Explain the four pillars of OOP.

Sample Answer: Encapsulation bundles data with the methods that operate on it and hides internal state, like a `#balance` private field only touched through `deposit/withdraw` methods. Abstraction exposes only what's needed and hides implementation, like calling `fetch()` without knowing the socket details underneath. Inheritance lets a class reuse and extend another's behavior, like `Admin` extends `User`. Polymorphism means the same method call behaves differently depending on the object, like `getAccess()` returning different results for `Admin` vs `Guest`.

Q2. Is JavaScript truly object-oriented?

Sample Answer: Not in the classical, class-based sense — JavaScript is prototype-based under the hood. ES6 class syntax is syntactic sugar over prototypes; every object still looks up methods along its prototype chain rather than through a rigid class hierarchy like Java. It supports OOP patterns, but its actual mechanism is different from classical OOP languages.

Q3. What is MVC?

Sample Answer: Model-View-Controller separates an application into three layers: the Model manages data and business logic, the View renders the UI, and the Controller handles incoming requests and coordinates between the two. This separation makes it possible to change the UI without touching business logic, or swap the data layer without touching the UI.

Q4. What are the SOLID principles — explain at least two.

Sample Answer: Single Responsibility means a class or module should have exactly one reason to change — a `UserController` shouldn't also handle email sending. Open/Closed means code should be open for extension but closed for modification — adding a new payment method shouldn't require editing existing payment logic, just adding a new implementation of a shared interface.

Q5. Does JavaScript support method overloading? What's the difference from overriding?

Sample Answer: No — JavaScript has no true method overloading; defining a second method with the same name simply replaces the first. Overriding is different: it happens in inheritance, where a subclass redefines a parent class's method, and the subclass's version runs instead — that's fully supported in JavaScript through prototypal inheritance and ES6 classes.

Q6. What makes code 'clean code' to you?

Sample Answer: Code that a stranger — or me, six months from now — can read and modify safely without asking me questions first. That means descriptive names over comments explaining bad names, small functions with a single clear purpose, and consistent patterns across the codebase rather than five different ways of doing the same thing.



10. Behavioral & HR Closing Questions

Use the STAR method (Situation, Task, Action, Result). Keep it honest — vague or overly rehearsed answers are easy for an interviewer to spot.

Q1. Tell me about a time you failed and what you learned.

Sample Answer: Situation: I once shipped a feature without testing an edge case where a list was empty. Task: my teammate found a crash in the demo the next day. Action: I fixed it immediately, then added a checklist of edge cases (empty, null, single item, very large input) to run through before calling anything 'done'. Result: I haven't had a similar bug make it to a demo since.

Q2. Describe a conflict with a teammate and how you resolved it.

Sample Answer: My teammate and I disagreed on whether to use REST or GraphQL for a project. Instead of arguing preferences, we listed our actual requirements — simple CRUD operations, small team, tight deadline — and REST was the clearer fit for those constraints. Deciding based on requirements rather than opinion resolved it quickly and respectfully.

Q3. How do you handle tight deadlines or pressure?

Sample Answer: I break the remaining work into the smallest shippable pieces and tackle the highest-risk or most uncertain part first, so if something's going to go wrong, I find out early rather than at the last hour. I also communicate early if something looks at risk, rather than staying quiet and hoping it resolves itself.

Q4. Do you have any questions for us?

Sample Answer: Yes — a few I'd genuinely ask: 'What does the onboarding process look like for a new engineer on this team?', 'What's a recent technical challenge the team worked through?', and 'What does success look like in this role after the first six months?' Always ask something specific to the team, never something answerable by a quick web search.

Q5. How do you keep learning new technologies?

Sample Answer: I try to build something small with a new tool rather than just reading about it — I learned [technology] by rebuilding a small piece of an existing project with it. I also follow a couple of engineering blogs and read real interview experiences like this one, which is how I know what companies are actually testing for right now.

Q6. Describe a time you had to explain something technical to a non-technical person.

Sample Answer: I explained to a non-technical teammate why our app felt slow by comparing the database to a library: without an index, finding a book means checking every shelf; with one, it's like having a catalog that points straight to the right shelf. Using an analogy instead of jargon got the point across immediately, and they trusted the fix because they understood why it mattered.



11. Glassdoor Interview Questions — Real Candidate Reports

Compiled and paraphrased from candidate-submitted interview reports across Glassdoor (Afford Medical Technologies, company id E5578938 — Backend, Frontend, Full Stack, and Software Developer listings), a GeeksforGeeks on-campus interview experience, and a candidate's public GitHub write-up of their full interview journey, all as of July 2026. Real experiences vary by batch, role, and interviewer — treat this as a pattern of what's commonly reported, not a guaranteed script. The final cluster below is clearly marked as simulated: reasonable follow-up questions built from these patterns, not verbatim reports.

Process & Company Facts, As Reported

Q1. What does the overall process and difficulty look like, according to real candidates?

Sample Answer: Candidates consistently describe three stages: resume shortlisting, a timed practical/task round, and a technical interview focused entirely on the project just built (sometimes followed by a separate HR or even CEO-level final round). Reported difficulty sits around 3.5–3.6 out of 5, with roughly a quarter to two-fifths of candidates rating their overall experience positively — several specifically describe the screening/task round as very demanding under a strict 3-hour limit.

Q2. How much time is actually given for the task round, by role?

Sample Answer: Multiple reports converge on the same split also referenced in the assessment brief: about 90 minutes for a backend-only or frontend-only track, and roughly double (2–3 hours) for the full-stack track, since it requires building both ends of the application against provided APIs and credentials.

Q3. What tech stack and role details do real candidates report?

Sample Answer: One detailed candidate write-up lists the team's stack as Node.js, NestJS, React, Next.js, Docker, Kubernetes, GoLang, and both SQL and NoSQL databases — confirming this is a genuinely polyglot, modern backend environment, not a single-framework shop. The same candidate reported a remote-first arrangement transitioning to onsite (Hyderabad) after some months, and a fresher CTC structured as a lower probation-period salary that increases after confirmation.

Q4. What do candidates say the technical interview actually focuses on?

Sample Answer: Almost entirely on the project just submitted — candidates are asked to share their screen and walk through their own code, explain specific implementation choices, and justify design decisions. Several describe the questioning as unexpectedly deep, with more than one interviewer present, each asking follow-up questions in turn — to the point where multiple candidates said the depth expected felt closer to an experienced hire's interview than a fresher's, even though they were freshers.

Frontend Conceptual Round — Real Reported Questions

Q5. Which state values did you use in your project, and why?

Sample Answer: I'd walk through each piece of state I introduced — for example, a list of items, a loading flag, and an error flag — and explain why each needed to be state (it changes over time and affects what's rendered) rather than a plain variable or a prop.

Q6. Explain useState and useEffect, and why useEffect needs a dependency array.

Sample Answer: useState holds a value across re-renders and gives a setter that triggers a new render. useEffect runs side effects after render and re-runs only when something in its dependency array changes — an empty array means 'run once on mount', and a missing array means 'run after every single render', which is almost always a bug waiting to happen.

Q7. What's a real use case for useMemo?

Sample Answer: Wrapping an expensive calculation — like filtering or sorting a large list — in useMemo so it only re-runs when its actual inputs change, instead of on every re-render caused by unrelated state updates elsewhere in the component.

**Q8. What happens with: `console.log(a)`; let `a = 10`;**

Sample Answer: This throws a `ReferenceError`, not `undefined`. `let` and `const` are hoisted but stay in the 'temporal dead zone' until their declaration line executes, so accessing them before that point is an error — this is different from `var`, which would just log `undefined`.

Q9. Can multiple `await` calls run at the same time?

Sample Answer: Not if they're awaited one after another in sequence — each one blocks the next line until it resolves. To run them concurrently, I'd kick off all the promises first without awaiting immediately, then use `Promise.all()` to wait for all of them together.

Q10. TypeScript: interface vs type, and why does type safety matter on a larger codebase?

Sample Answer: Both describe an object's shape; I'd use interface for extendable object contracts and type for unions or more complex type expressions. On a larger codebase, type safety catches an entire class of bugs at compile time — like passing a string where a number was expected — before they ever reach production.

Q11. Explain the CSS box model, and the difference between block and inline display.

Sample Answer: The box model is content, then padding, then border, then margin, from the inside out. A block element takes the full available width and starts on a new line (like a `div`); an inline element only takes as much width as its content and sits in the flow of surrounding text (like a `span`).

Q12. How does routing work in Next.js, and why didn't you use it for this project?

Sample Answer: Next.js uses file-based routing — a file's path inside the pages or app directory becomes its URL automatically. I didn't use it here because [reasoning — e.g. the project didn't need server-side rendering or SEO, and plain React with a lighter router was a faster fit for the timeline].

Q13. Axios vs the native fetch API — what's the real difference?

Sample Answer: Both send HTTP requests, but Axios automatically parses JSON, has built-in request/response interceptors, and treats non-2xx responses as errors by default — fetch requires manually checking `response.ok` and calling `.json()` yourself, so it's more boilerplate for the same result.

Q14. When would you use `<Link>` instead of a plain `<a>` tag?

Sample Answer: `<Link>` (in React Router or Next.js) does client-side navigation without a full page reload, preserving app state and loading faster. A plain `<a>` tag triggers a full browser navigation — correct for leaving the app entirely, but wasteful for moving between pages inside it.

Backend Conceptual Round — Real Reported Questions**Q15. What caching methods have you used or would you use — in-memory vs Redis?**

Sample Answer: In-memory caching (a plain object or `Map` in the running process) is fastest but only lives on one server and resets on restart. Redis is a separate, shared cache that every server instance can read from, which matters the moment there's more than one server running.

Q16. How would you store and reuse data fetched from an external API?

Sample Answer: I'd cache the response with a short time-to-live suited to how often the source data actually changes, and always design my own function so it checks the cache first and only calls the external API on a miss — protecting both my own performance and the external API's rate limits.

Q17. Walk me through your REST route and endpoint design for a new resource.

Sample Answer: I'd use a plural resource noun as the base path, HTTP methods to express the action instead of the URL, and consistent nesting for relationships — for example `/journals` for the collection and `/journals/:id` for a single entry, rather than inventing a new verb-based endpoint per action.

Q18. How do you handle a Bearer token in request headers, and why is storing it in a client-side `.env` risky?



Sample Answer: I'd attach it as an Authorization: Bearer <token> header on each request. Anything in a client-side .env file ships inside the built frontend bundle, which anyone can inspect in the browser — secrets belong on the server, or in a token that's short-lived and scoped, never a long-lived secret shipped to the client.

Q19. Write a query using JOIN, GROUP BY, and ORDER BY together.

Sample Answer: For example: counting notifications per student and listing the busiest students first — `SELECT s.name, COUNT(n.id) AS total FROM students s JOIN notifications n ON n.studentID = s.studentID GROUP BY s.name ORDER BY total DESC`. I'd explain that the JOIN links the tables, GROUP BY collapses rows per student, and ORDER BY sorts the aggregated result.

Machine Coding Rounds — Real Reported Task Formats

Q20. What does a typical frontend machine-coding prompt look like?

Sample Answer: One reported example: clone a given temperature-converter UI design exactly, with the conversion formula supplied, in about 25 minutes — with no internet search and no editor extensions allowed. Another: build an expense tracker — first just a list with a running total (about 20 minutes), then extend it with a form and localStorage persistence (about 15 more minutes).

Q21. What does a typical backend machine-coding prompt look like?

Sample Answer: One reported example: build CRUD APIs for a journal entry (title, description, tags, date, and a mood field of neutral/happy/sad), including endpoints to retrieve entries filtered by date and by mood — in about 30 minutes, in any backend stack of your choice.

Q22. Is SQL / query performance actually asked about outside the take-home?

Sample Answer: Yes — one candidate specifically described a full-stack round with seven phases to complete in three hours, where part of the evaluation was SQL query optimization alongside documentation and a React component that consumes a provided API. This lines up directly with the Stage 3 slow-query problem covered earlier in this document — it's worth treating as near-certain to come up in some form.

Other Reported Topics — HTTP Codes & DSA

Q23. What HTTP/REST-related questions come up often?

Sample Answer: The difference between GET, POST, PUT, and PATCH, and when each is the correct choice; and knowing common HTTP status codes (candidates specifically mention 201, 202, and 404) and what each communicates to a client, not just that they exist.

Q24. Are plain DSA questions asked even in a frontend-track interview?

Sample Answer: Yes — candidates report being asked to implement or reason about a linked list even when interviewing primarily for a frontend role, so core data structure fundamentals are fair game regardless of which track you're on.

Simulated Likely Follow-Ups (Derived, Not Directly Reported)

Q25. If you used Promise.all() to parallelize requests, what happens if one of them fails?

Sample Answer: By default, Promise.all() rejects immediately if any single promise rejects, even if the others would have succeeded. If partial success is acceptable, I'd use Promise.allSettled() instead, which waits for every promise and reports each one's individual outcome.

Q26. Why might a team choose NestJS over plain Express as it grows?

Sample Answer: NestJS enforces a consistent, modular structure (modules, controllers, services) out of the box with built-in dependency injection, which keeps a large codebase organized as more engineers join — plain Express gives more freedom but also more room for every developer to structure things differently.

Q27. When would you reach for Docker alone versus Docker plus Kubernetes?



Sample Answer: Docker packages an app and its dependencies into a consistent container; that's often enough for a small team running a handful of services. Kubernetes becomes worth the added complexity once you need to orchestrate many containers automatically — scaling instances up and down, restarting failed ones, and load balancing between them.

Q28. Your machine-coding round didn't allow internet access or extensions — how do you practice for that?

Sample Answer: I'd rehearse common tasks (a CRUD API, a small form-driven UI, a filter/search feature) from memory on a timer, using only a plain editor with no autocomplete plugins, so the constraint isn't a surprise on the actual day.

Q29. If asked to defend a piece of code you didn't write from scratch (a common pattern/snippet), what should you say?

Sample Answer: Be honest that it's a common, well-known pattern rather than claiming to have invented it from nothing — then demonstrate real understanding by explaining exactly why it works and how you'd modify it if the requirements changed slightly.

Q30. How would you prepare differently after reading these real reports?

Sample Answer: Two things stand out across almost every report: first, be ready to defend your own submitted code in real depth, out loud, to more than one interviewer — not just describe what it does but why you built it that way. Second, don't neglect 'plain JavaScript, TypeScript, and React basics' in favor of only studying the bigger system-design stages — several candidates who found the process easy specifically credited solid fundamentals in async JS, HTTP methods, and core React hooks.



12. Final Round Logistics & Live Coding Prep

Based on Affordmed's official interview-scheduling message to shortlisted candidates. This round is confirmed live and one-on-one (not just a recorded submission review), runs concurrently across candidates in ~30-minute slots, and is conducted entirely from the candidate's own institution. Expect live coding, deeper follow-ups on the assessment at medium-to-hard difficulty, and direct questions from the standard B.Tech CSE curriculum — not just the web-dev stack used in the submission.

On-the-Day Logistics Checklist

- Confirm the exact date/time as soon as it's announced, and treat the whole scheduled window as blocked — interviews run concurrently, so your slot could be called earlier than expected.
- Join only from a laptop/computer on a stable, wired-if-possible connection — mobile phone participation is not allowed.
- Take the call from a quiet, distraction-free room on campus — this is mandatory regardless of day or time, including weekends.
- Test your microphone beforehand — audio issues can result in being skipped in favor of the next candidate, since the schedule is fixed in advance.
- Have your own submitted code open and ready in your local development environment (VS Code or equivalent) — online IDEs are explicitly not permitted during the interview.
- Request read access to the test document for your specific track (backend/frontend/full-stack) ahead of time, so you're not doing this at the last minute.
- Re-read your own submitted assessment question and your own solution in full before the interview — identify, in advance, which parts you're confident about and which parts you know are weaker or incomplete.
- Stay actively watching the group/channel for turn updates, and be ready the moment the candidate before you is called in.

What This Confirms About the Interview Format

Q1. Is this interview just a discussion of my submission, or will there be live coding?

Sample Answer: Treat it as both. The message explicitly asks candidates to revise Data Structures, Algorithms, and REST APIs — that's a strong signal of live, on-the-spot coding in addition to a walkthrough of the assessment. Expect to be asked to write or trace through code in real time, not just explain what you already submitted.

Q2. Why does the message mention the 'standard CSE curriculum for a B.Tech in India' specifically?

Sample Answer: Because this round isn't limited to the web-development stack used in the assessment — it can reach into core computer science subjects like Operating Systems, Computer Networks, and DBMS theory that every B.Tech CSE student covers, regardless of which stack they used for the take-home. Don't prepare only your project stack and skip core CS fundamentals.

Q3. What difficulty level should I actually prepare for?

Sample Answer: Medium to hard, not warm-up level. This is a final technical round after you've already cleared a demanding take-home assessment, so questions build on top of what you've already proven you can do — expect deeper follow-ups, tighter time pressure, and less hand-holding than the earlier rounds.

Live Coding Practice — Medium-to-Hard Problems

Q4. Given a live stream of events, each with a priority weight and a timestamp, maintain and return the top-K highest-priority items at any point in time.

Sample Answer: Use a min-heap of size K keyed on priority: for each new event, push it in; if the heap exceeds size K, pop the smallest. This keeps insertion at $O(\log K)$ instead of re-sorting the whole stream, and directly mirrors the Priority Inbox problem from the assessment — be ready to code this from scratch, not just describe it.



Q5. Given tasks with a duration and an impact score, and a fixed time budget, find the maximum total impact achievable.

Sample Answer: Classic 0/1 Knapsack: build a DP table where $dp[i][t]$ is the best impact achievable using the first i tasks within t time units, filling it from $dp[i-1][t]$ (skip the task) vs $dp[i-1][t-\text{duration}] + \text{impact}$ (take the task). State the $O(n \times \text{budget})$ time and space complexity, and be ready to explain why a greedy pick-highest-impact-first approach fails.

Q6. Given a list of notification timestamps, count how many fall within any rolling 24-hour window.

Sample Answer: Sort the timestamps, then use a sliding window / two-pointer approach: advance the right pointer forward, and shrink the left pointer whenever the window exceeds 24 hours, tracking the maximum window size seen — $O(n \log n)$ for the sort, $O(n)$ for the scan, instead of an $O(n^2)$ brute force comparison of every pair.

Q7. Design and implement a simple LRU (Least Recently Used) cache with get and put in $O(1)$.

Sample Answer: Combine a hash map (key to node) with a doubly linked list (to track recency order). On get, move the accessed node to the front; on put, insert at the front and evict the node at the back if over capacity. This is a very common 'medium-hard' live coding ask and connects directly to the caching concepts already covered in this document.

Q8. Given a batch of API request logs, count how many requests hit each endpoint per minute.

Sample Answer: Truncate each timestamp down to the minute, then use a hash map keyed on (endpoint, minute) to accumulate counts in a single pass — $O(n)$ time. Mention that in a real system this is exactly the kind of aggregation a monitoring/logging pipeline (like the Logging Middleware from the assessment) would need to support.

Q9. Write a debounce function in JavaScript from scratch.

Sample Answer: A debounce wraps a function so it only actually runs after a pause in calls — store a timer id in closure, clear it on every new call, and set a fresh `setTimeout` each time; the wrapped function only fires once the delay passes without another call. Be ready to explain a real use case, like avoiding an API call on every keystroke in a search box.

Q10. Live-code a paginated GET endpoint for a resource, supporting both limit/offset and cursor-based approaches — and explain the trade-off.

Sample Answer: Limit/offset is simple (`LIMIT :limit OFFSET :offset`) but gets slower and can skip/duplicate rows if data changes between pages at scale. Cursor-based pagination (`WHERE id > :lastSeenId ORDER BY id LIMIT :limit`) stays fast and stable at scale since it never has to skip rows, at the cost of not supporting 'jump to page 7' directly.

Q11. Given a batch of incoming notifications, detect and remove exact duplicates before they reach the database.

Sample Answer: Build a composite key per notification (e.g. `studentID + type + message + a time bucket`), and use a Set to track keys already seen in the batch — $O(n)$ to de-duplicate. Mention that this only catches duplicates within the current batch; true idempotency across separate requests needs a persisted key, which ties back to the Stage 5 idempotency discussion earlier in this document.

Standard CSE Curriculum Quick-Hits — Operating Systems

Q12. What's the difference between a process and a thread?

Sample Answer: A process is an independent program in execution with its own memory space; a thread is a lighter-weight unit of execution within a process that shares that process's memory with other threads. Threads are cheaper to create and switch between, but shared memory means they need careful synchronization to avoid race conditions.

Q13. What is a deadlock, and what are the four necessary conditions for one to occur?

Sample Answer: A deadlock is when two or more processes are stuck waiting on each other forever. The four conditions are: mutual exclusion (a resource can only be held by one process), hold-and-wait (holding one resource while waiting for another), no preemption (resources can't be forcibly taken away), and circular wait (a cycle of processes each waiting on the next). Breaking any one of these prevents deadlock.

**Q14. Why does an operating system need virtual memory and paging?**

Sample Answer: Virtual memory gives each process the illusion of a large, contiguous, private address space, even though physical RAM is limited and shared. Paging breaks memory into fixed-size pages so the OS can load only the pages currently needed into physical RAM and swap out the rest, letting the system run more and larger processes than physical memory alone would allow.

Q15. What's the difference between preemptive and non-preemptive CPU scheduling?

Sample Answer: In preemptive scheduling, the OS can interrupt a running process and switch to another before it finishes its turn (e.g. Round Robin, Priority Scheduling with preemption) — better for responsiveness. In non-preemptive scheduling, once a process starts running, it keeps the CPU until it finishes or voluntarily yields (e.g. First-Come-First-Served) — simpler, but a long process can block everything behind it.

Standard CSE Curriculum Quick-Hits — Computer Networks**Q16. Briefly explain the OSI model or the TCP/IP layers.**

Sample Answer: The simplified version most interviews care about: Application (HTTP, the actual data), Transport (TCP/UDP — how it's delivered), Network (IP — how it's routed/addressed), and Link (how it physically travels). Being able to say which layer a concept belongs to — e.g. 'TCP is transport layer' — matters more than reciting all seven OSI layers by name.

Q17. What's the difference between TCP and UDP, and when would you use each?

Sample Answer: TCP is connection-oriented and guarantees ordered, reliable delivery via acknowledgements and retransmission — correct for anything where losing data matters, like an API request. UDP is connectionless and doesn't guarantee delivery or order, but has much lower overhead — correct for real-time use cases like video calls or live game state, where a dropped packet is better than a delayed one.

Q18. What happens when you type a URL into a browser and press enter?

Sample Answer: In order: the browser checks its cache then does a DNS lookup to resolve the domain to an IP address; it opens a TCP connection to that IP (with a TLS handshake if HTTPS); it sends an HTTP request; the server processes it and sends back an HTTP response; and the browser parses the HTML/CSS/JS and renders the page, making further requests for any additional assets along the way.

Q19. What's the difference between HTTP and HTTPS?

Sample Answer: HTTPS is HTTP layered on top of TLS/SSL encryption — the request and response are encrypted in transit, so a third party intercepting the traffic can't read or tamper with it, and the client can verify the server's identity via its certificate. Functionally the methods and status codes are identical; the difference is entirely about transport security.